Reviewer Pack — Free Entropy Lower Bound on Disjointness Communication Compl…

Entry 50dd985f0904 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 14:30:33 UTC

Free Entropy Lower Bound on Disjointness Communication Complexity

Entry ID: `50dd985f0904` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 14:30:33 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

> For any n×n matrix M representing a DISJOINTNESS instance, the free entropy $\phi(M) = -\int \log(\det(M + tI))\, dt$ satisfies $\phi(M) \geq \Omega(\sqrt{n})$ with equality iff M is a rank-1 matrix. The conjecture quantifies the exponential gap between $\phi(M)$ and the communication complexity of M.

Rationale (proposer's reasoning):

> Free entropy captures spectral properties of random matrices, which may encode the entanglement structure of DISJOINTNESS instances. The integral over det(M + tI) relates to the matrix's eigenvalue distribution, which could mirror the information-theoretic cost of coordination in communication protocols.

Taxonomy category: `SOS_HIERARCHY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `5823b2599d2ffc30`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    n = 30
    M = [[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)]

    def log_det(M_t):
        U = gaussian_elimination(M_t)
        det = 1.0
        for i in range(n):
            det *= U[i][i]
        return math.log(abs(det))

    def f(t):
        M_t = [[M[i][j] + t * delta(i, j) for j in range(n)] for i in range(n)]
        return log_det(M_t)

    def adaptive_quadrature(f, a, b):
        n = 10
        s = 0.5 * (f(a) + f(b))
        h = (b - a) / (2 * n)
        for k in range(1, n):
            s += f(a + (2 * k - 1) * h)
        return s * h

    phi_M = adaptive_quadrature(f, -10, 10) / (2 * math.pi)

    metric_value = phi_M / math.sqrt(n)
    conjecture_holds = metric_value >= 0.8
    counterexample = "" if conjecture_holds else "phi(M) < 0.7√n"

    return {
        "metric_name": "free_entropy_ratio",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
```

```
        "counterexample": counterexample
    }

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i + argmax(abs(row[i]) for row in A[i:])
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            A[j] = [A[j][k] - factor * A[i][k] for k in range(n)]
    return A

def argmax(iterable):
    return max(range(len(iterable)), key=iterable.__getitem__)

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean:.6f} std={std_dev:.6f} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"phi(M) < 0.7√n\" first_failing_seed={first_fail
    else:
        print("RESULT: INCONCLUSIVE insufficient data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_963dfb76.py", line 74, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_963dfb76.py", line 41, in run_trial
    phi_M = adaptive_quadrature(f, -10, 10) / (2 * math.pi)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_963dfb76.py", line 35, in adaptive_quad
    s = 0.5 * (f(a) + f(b))
               ^^^^
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_963dfb76.py", line 30, in f
    M_t = [[M[i][j] + t * delta(i, j) for j in range(n)] for i in range(n)]
                          ^^^^^
NameError: name 'delta' is not defined
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed due to undefined 'delta' function, preventing evaluation of the conjecture's validity. | next: Define the Kronecker delta function in the code or debug the matrix construction logic

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111407
2	propose	ollama_remote	qwen3:8b	0	0	29541
3	preregistration	ollama_remote	qwen3:8b	0	0	24342
4	novelty	ollama_remote	qwen3:8b	0	0	20717
5	novelty	ollama_remote	qwen3:8b	0	0	14856
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12796
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9780
8	judge	ollama_remote	qwen3:8b	0	0	17357

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 240795 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/50dd985f0904.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/50dd985f0904.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/50dd985f0904.tar.gz` (if generated)